PHIẾU THỰC HÀNH — 09/10/2026

Họ tên: Đỗ Văn Bình **MSSV:** N24DCDT009 **Lớp:** D24CQDT01-N

GitHub: https://github.com/dovanbinh/CTDL_GT_Julia_2026_10_09 **Commit SHA:** fd1aa645363b6c4744ffe8966f5888998f59b82b

1. Mô tả một luồng ứng dụng

Ghi mục tiêu, đầu vào, đầu ra và sơ đồ truyền dữ liệu giữa các chức năng.

• Mục tiêu: Xây dựng hệ thống điều phối giao đơn hàng tự động trong khuôn viên trường bằng ngôn ngữ Julia. Từ dữ liệu bản đồ và danh mục đơn hàng, tính toán tuyến đường ngắn nhất (khứ hồi từ kho trung tâm điểm 1), áp dụng quy hoạch động (Knapsack 0/1) để chọn tập đơn tối đa hóa lợi ích trong ngân sách thời gian cho trước, quản lý tiến trình giao hàng và hoàn tác bằng hàng đợi FIFO và ngăn xếp LIFO, xuất kết quả chi tiết ra tệp CSV.

• Đầu vào:

1. don_hang.csv: Danh mục đơn hàng gồm mã đơn (id), điểm giao, thời gian phục vụ, lợi ích.
2. ban_do.csv: Đồ thị vô hướng không trọng số gồm các cạnh nối giữa các điểm trong khuôn viên.
3. Ngân sách thời gian B (với bộ mẫu MINI001 là B = 8; bộ mở rộng cá nhân MSSV N24DCDT009 là B = 30).

• Đầu ra: Kế hoạch giao hàng tối ưu, đường đi chi tiết và chi phí của từng đơn, trạng thái giao/hoàn tác thể hiện qua Queue và Stack, tệp ket_qua.csv ghi nhận đầy đủ 9 trường thông tin cho toàn bộ đơn hàng.

• Sơ đồ truyền dữ liệu giữa các chức năng:

[don_hang.csv, ban_do.csv] --> (read_data: nạp danh mục đơn & cạnh đồ thị)
--> (merge_sort: chuẩn hóa danh sách kề & sắp xếp danh mục theo ID tăng)
--> (bfs_routes: tìm đường đi ít cạnh nhất từ kho 1, tính thời gian khứ hồi $\tau_i = 2*h_i + s_i$)
--> (knapsack: quy hoạch động ba lô 0/1 chọn tập đơn tối ưu lợi ích trong ngân sách B)
--> (CircularQueue & Stack: nạp đơn chọn theo thứ tự ưu tiên lợi ích/thời gian vào hàng đợi FIFO)
--> (deliver! / undo!: giao đơn lấy từ đầu Queue đẩy vào Stack, hoàn tác pop từ

Stack trả lại đầu Queue)

--> (export_result: xuất tệp kết quả ket_qua.csv chụp lại toàn bộ trạng thái).

2. Bảng truy vết đường và thời gian

ID	Đường lượt đi	Số cạnh h	2h + phục vụ
1	1 -> 2	1	2×1 + 1 = 3
2	1 -> 2 -> 4	2	2×2 + 1 = 5
3	1 -> 2 -> 4 -> 6	3	2×3 + 1 = 7

3. Minh chứng bốn hàm

NV1 · Hoàn thiện merge_sort(a, before)

Ý tưởng/điều kiện áp dụng: Áp dụng chiến lược Chia để trị (Divide and Conquer). Dãy được chia đôi đệ quy cho đến khi độ dài ≤ 1 (điều kiện dừng). Sau đó, hai nhánh đã có thứ tự được trộn (merge) tuần tự vào vector mới theo hàm so sánh before. Để đảm bảo tính ổn định (stable sort), khi hai phần tử hòa theo before, phần tử nhánh trái luôn được ưu tiên chọn trước. Mảng đầu vào được giữ nguyên vẹn, không chỉnh sửa trực tiếp.

Lời gọi và kết quả kiểm thử: Lệnh kiểm tra: julia TH_2026_10_09_Kiem_thu.jl --phase=sort. Kết quả: ĐẠT 8/8 phép kiểm tra (xử lý chính xác mảng rỗng [], mảng 1 phần tử [7], sắp tăng [7,2,5,1] -> [1,2,5,7], sắp giảm với phần tử trùng lặp [2,1,2,3] -> [3,2,2,1], bảo toàn tính ổn định trên cặp khóa-giá trị, và sắp xếp danh mục đơn Order theo ID).

Chi phí thời gian/bộ nhớ: Thời gian: O(n log n) trong mọi trường hợp (tốt nhất, trung bình, xấu nhất) do cây đệ quy có độ sâu $\lceil \log_2 n \rceil$ và mỗi tầng tốn O(n) phép so sánh/trộn. Bộ nhớ phụ: O(n) cho mảng đệm kết quả trộn và O(log n) cho ngăn xếp đệ quy.

NV2 · Hoàn thiện binary_search(a::Vector{Order}, id::Int)

Ý tưởng/điều kiện áp dụng: Tiền điều kiện bắt buộc: Danh mục đơn hàng Order phải được sắp xếp tăng dần theo trường id và các id phải duy nhất. Duy trì khoảng tìm kiếm [lo, hi], ở mỗi bước tính chỉ số giữa mid = lo + div(hi - lo, 2). So sánh a[mid].id với id: nếu bằng thì trả về bản ghi Order; nếu a[mid].id < id thì thu hẹp sang nửa phải (lo = mid + 1); nếu lớn hơn thì thu hẹp sang nửa trái (hi = mid - 1). Khi lo > hi thì dừng và trả về nothing.

Lời gọi và kết quả kiểm thử: Lệnh kiểm tra: julia TH_2026_10_09_Kiem_thu.jl --phase=search. Kết quả: ĐẠT 5/5 phép kiểm tra (tìm kiếm trên mảng rỗng trả nothing; tìm phần tử ở đầu dãy ID 2 và cuối dãy ID 7 trả đúng bản ghi; tìm phần tử giữa ID 4 trả đúng Order có benefit = 8; tìm mã không tồn tại ID 3 và ID 99 trả về nothing).

Chi phí thời gian/bộ nhớ: Thời gian: O(log n) do không gian tìm kiếm giảm một nửa sau mỗi phép so sánh; trường hợp tốt nhất O(1) nếu phần tử cần tìm ở ngay giữa. Bộ nhớ phụ: O(1) vì cài đặt lặp, chỉ sử dụng ba biến nguyên lo, mid, hi.

NV3 · Hoàn thiện bfs_routes(graph, source=1)

Ý tưởng/điều kiện áp dụng: Áp dụng thuật toán duyệt theo chiều rộng (BFS) trên đồ thị vô hướng không trọng số để tìm đường đi có số cạnh ít nhất từ đỉnh nguồn source = 1. Sử dụng hàng đợi CircularQueue (FIFO). Khởi tạo distance = -1, parent = 0. Đặt distance[source] = 0 và enqueue nguồn. Khi duyệt đỉnh u, với mỗi đỉnh kề v chưa thăm (distance[v] == -1), gán distance[v] = distance[u] + 1, parent[v] = u và lập tức enqueue v (đánh dấu ngay khi đưa vào hàng đợi để không bị lặp đỉnh). Dựng đường đi routes[v] bằng cách lần ngược mảng parent từ v về nguồn rồi đảo ngược mảng.

Lời gọi và kết quả kiểm thử: Lệnh kiểm tra: julia TH_2026_10_09_Kiem_thu.jl --phase=bfs. Kết quả: ĐẠT 7/7 phép kiểm tra (khoảng cách từ nguồn 1 đến các đỉnh đúng bằng [0, 1, 1, 2, 2, 3]; các tuyến đường hợp lệ và khớp nút cha; chi phí thời gian τ cho ba đơn đúng bằng [3, 5, 7]; đỉnh cô lập không tới được trả distance = -1 và routes rỗng; hỗ trợ đúng nguồn khác source = 4).

Chi phí thời gian/bộ nhớ: Thời gian: O(V + E) cho quá trình duyệt BFS trên danh sách kề và dựng tuyến đường. Bộ nhớ phụ: O(V) cho hàng đợi CircularQueue, mảng distance và parent. Dung lượng lưu trữ kết quả routes tối đa là $O(V^2)$.

NV4 · Hoàn thiện knapsack(orders, costs, budget)

Ý tưởng/điều kiện áp dụng: Áp dụng Quy hoạch động giải bài toán Ba lô 0/1 (0/1 Knapsack) để tối đa hóa tổng lợi ích trong giới hạn ngân sách thời gian B. Gọi F(i, b) là lợi ích lớn nhất khi xét i đơn hàng đầu tiên với ngân sách b ($0 \le i \le n$, $0 \le b \le B$). Trong Julia, lưu tại table[i+1, b+1]. Công thức truy hồi:

- Nếu $c_i > b$: table[i+1, b+1] = table[i, b+1]
- Nếu $c_i \le b$: table[i+1, b+1] = max(table[i, b+1], table[i, b - c_i + 1] + v_i). Khi hai phương án bằng nhau, ưu tiên giữ phương án không lấy đơn i.

Truy vết từ ô table[n+1, budget+1] ngược về i = 1: nếu table[i+1, curr_b+1] != table[i, curr_b+1] thì ghi nhận id đơn i và giảm curr_b -= c_i.

Lời gọi và kết quả kiểm thử: Lệnh kiểm tra: julia TH_2026_10_09_Kiem_thu.jl --phase=dp. Kết quả: ĐẠT 12/12 phép kiểm tra (kết quả DP khớp tuyệt đối với oracle vét cạn độc lập tại các ngân sách 0, 5, 8, 15; với ngân sách B = 8 bộ mini chọn chính xác tập ID {1, 2} có cost = 8, value = 14; bảng DP có hàng/cột cơ sở 0; từ chối ngân sách âm; chứng minh tham lam thất bại trước DP trong phản ví dụ; bất biến thứ tự dữ liệu đầu vào).

Chi phí thời gian/bộ nhớ: Thời gian: O(n × B) để điền bảng quy hoạch động và O(n) cho bước truy vết. Bộ nhớ phụ: O(n × B) để lưu trữ bảng table kích thước (n+1) × (B+1).

4. Quyết định chọn đơn

Điền ngân sách, tập chọn, tổng thời gian/lợi ích. Giải thích một ô DP và một bước truy vết; đối chiếu với vét cạn.

• Ngân sách thời gian: B = 8.
• Tập đơn được chọn: ID {1, 2} (Đơn 1 và Đơn 2).
• Tổng thời gian thực hiện: cost = $\tau_1 + \tau_2 = 3 + 5 = 8$ (sử dụng tối đa 100% ngân sách 8).
• Tổng lợi ích đạt được: value = $v_1 + v_2 = 6 + 8 = 14$.
• Giải thích ô DP F(2, 8):
  - Tại bước xét đơn 2 ($\tau_2 = 5$, $v_2 = 8$) với ngân sách b = 8:
    + Phương án 1 (không chọn đơn 2): Lấy giá trị tối ưu của đơn 1 với ngân sách 8: F(1, 8) = 6.
    + Phương án 2 (chọn đơn 2): Lấy lợi ích của đơn 2 cộng giá trị tối ưu của đơn 1 với ngân sách còn lại (8 - 5 = 3): v_2 + F(1, 3) = 8 + 6 = 14.
    + So sánh: F(2, 8) = max(F(1, 8), v_2 + F(1, 3)) = max(6, 14) = 14.
  - Vị trí trong ma trận Julia: Ô F(2, 8) nằm tại tọa độ table[2+1, 8+1] = table[3, 9] vì Julia đánh chỉ số từ 1.
• Giải thích bước truy vết:
  - Bắt đầu từ ô cuối table[4, 9] (i = 3, b = 8): table[4, 9] = 14 == table[3, 9] = 14 --> Không chọn đơn 3, giữ b = 8.
  - Tại i = 2, b = 8: table[3, 9] = 14 != table[2, 9] = 6 --> Chọn đơn 2! Thêm ID 2, ngân sách còn b = 8 - τ_2 = 8 - 5 = 3.
  - Tại i = 1, b = 3: table[2, 4] = 6 != table[1, 4] = 0 --> Chọn đơn 1! Thêm ID 1,

ngân sách còn b = 3 - τ_1 = 3 - 3 = 0.

- Kết quả tập chọn: [1, 2].

• Đối chiếu với vét cạn (Brute-force) trên $2^3 = 8$ tập con:

- ∅: cost = 0, value = 0
- {1}: cost = 3, value = 6
- {2}: cost = 5, value = 8
- {3}: cost = 7, value = 10
- {1, 2}: cost = 3 + 5 = 8 ≤ 8, value = 6 + 8 = 14 (HỢP LỆ VÀ TỐI ƯU NHẤT)
- {1, 3}: cost = 3 + 7 = 10 > 8 (Loại do vượt ngân sách)
- {2, 3}: cost = 5 + 7 = 12 > 8 (Loại do vượt ngân sách)
- {1, 2, 3}: cost = 3 + 5 + 7 = 15 > 8 (Loại do vượt ngân sách)

--> Giá trị tối ưu của Quy hoạch động (14) hoàn toàn trùng khớp với vét cạn.

5. Giao và hoàn tác

Trạng thái	Queue	Stack	Đã dùng	Đã nhận
Sau lập kế hoạch	[1, 2]	[]	0	0
Sau giao một đơn	[2]	[1]	3	6
Sau hoàn tác	[1, 2]	[]	0	0
Sau giao hết	[]	[1, 2]	8	14

6. Đầu ra và nhận xét

Đường dẫn CSV kết quả: data/MINI001/ket_qua.csv (và data/N24DCDT009/ket_qua.csv) **Link notebook:** TH_2026_10_09_Pluto.jl

Lỗi đã phát hiện và cách sửa: 1. Lỗi lệch chỉ số ma trận quy hoạch động trong Julia (1-based index): Hàng và cột cơ sở đại diện cho i = 0 và b = 0. Do đó, kích thước ma trận phải là (n+1) × (budget+1), trạng thái F(i, b) được lưu tại table[i+1, b+1], và khi tham chiếu phương án lấy đơn ở hàng trước với ngân sách còn lại b - c_i phải là table[i, b - c_i + 1]. Đã sửa đúng công thức, khắc phục lỗi truy cập ngoài biên và tính sai giá trị.

2. Lỗi thời điểm đánh dấu đỉnh trong BFS: Nếu chỉ đánh dấu đã thăm khi lấy đỉnh ra khỏi hàng đợi (dequeue), một đỉnh có thể bị nhiều đỉnh kề đưa vào hàng đợi lặp lại nhiều lần. Cách sửa: gán distance[v] và parent[v] ngay khi phát hiện đỉnh v chưa thăm trước khi enqueue!(q, v).

3. Lỗi tra cứu chi phí: costs là Dict tra theo mã ID đơn hàng (costs[o.id]), không phải tra theo chỉ số mảng i.

Một chức năng thay đổi ảnh hưởng đến chức năng sau như thế nào? Các chức năng trong chương trình tạo thành một chuỗi luồng dữ liệu (pipeline) phụ thuộc nhân quả chặt chẽ:

- Nếu cấu trúc bản đồ thay đổi (thêm/bớt cạnh), thuật toán BFS (bfs_routes) sẽ tìm ra các đường đi ngắn nhất khác, dẫn tới số cạnh h và chi phí thời gian thực hiện $\tau = 2h + s$ của từng đơn thay đổi.
- Chi phí τ thay đổi làm thay đổi trực tiếp trọng số đầu vào của thuật toán quy hoạch động (knapsack), làm thay đổi bảng DP và dẫn tới tập đơn được chọn tối ưu thay đổi hoàn toàn (ví dụ: mất cạnh (2,4) khiến chi phí đơn 2 tăng từ 5 lên 9, khiến kế hoạch tối ưu chuyển sang chọn đơn 3 thay vì đơn 1+2).
- Tập đơn chọn thay đổi sẽ kéo theo các phần tử nạp vào hàng đợi CircularQueue thay đổi, thay đổi trình tự giao hàng, lịch sử hoàn tác và kết quả cuối cùng xuất ra file ket_qua.csv.

Câu hỏi cuối buổi

Q1. Vì sao không được dùng dãy đã sắp giảm theo lợi ích/τ để tìm kiếm nhị phân theo ID? Nêu tiền điều kiện của hàm tìm kiếm.

• Vì sao không được dùng dãy sắp giảm theo lợi ích/τ để tìm kiếm nhị phân theo ID:

Tìm kiếm nhị phân (Binary Search) hoạt động dựa trên tiên đề mảng đã được sắp thứ tự đơn điệu theo đúng khóa tìm kiếm (ở đây là trường ID). Khi so sánh giá trị khóa tại vị trí giữa a[mid].id với id cần tìm, thuật toán mới có thể khẳng định chắc chắn loại bỏ nửa trái hay nửa phải. Nếu dãy được sắp giảm theo tỷ số ưu tiên (lợi ích/τ), các giá trị id sẽ phân bố lộn xộn, không tăng dần. Do đó việc dựa vào a[mid].id < id để loại bỏ nửa mảng sẽ loại bỏ sai nhánh chứa phần tử cần tìm, dẫn đến kết quả sai hoàn toàn (trả về nothing hoặc tìm sai đơn).

• Tiền điều kiện của hàm tìm kiếm binary_search:

Mảng danh mục Order đầu vào phải được sắp xếp tăng dần nghiêm ngặt theo trường id (tức a[i].id < a[i+1].id với mọi i), các mã id phải là duy nhất, không trùng lặp và hàm so sánh phải nhất quán.

Q2. Với đơn ID 2 tới điểm 4, phục vụ 1 và bản đồ mẫu, giải thích từng thành phần của thời gian thực hiện τ. BFS có tiếp tục cho thời gian ngắn nhất nếu các cạnh mang thời gian khác nhau không?

• Giải thích từng thành phần của thời gian thực hiện τ với đơn ID 2 (tới điểm 4, phục vụ 1):

Công thức tính: $\tau_i = 2 \times h_i + s_i$.

- $h_2 = 2$: Số cạnh của tuyến đường đi ngắn nhất một chiều từ kho trung tâm 1 tới điểm giao 4 (tuyến: 1 -> 2 -> 4), được tìm bởi thuật toán BFS.
- Hệ số $2 \times h_2 = 2 \times 2 = 4$: Thời gian di chuyển của chuyến đi khứ hồi (chuyến đi từ kho 1 tới điểm 4 mất 2 đơn vị thời gian, chuyến quay về từ điểm 4 về kho 1 mất 2 đơn vị thời gian, mỗi cạnh tốn 1 đơn vị thời gian).
- $s_2 = 1$: Thời gian dừng lại để phục vụ / bàn giao đơn hàng tại điểm giao 4.

--> Tổng thời gian thực hiện của chuyến giao đơn 2 là: $\tau_2 = 2 \times 2 + 1 = 5$ đơn vị thời gian.

• BFS có tiếp tục cho thời gian ngắn nhất nếu các cạnh mang thời gian khác nhau không:

KHÔNG. Thuật toán BFS chỉ đảm bảo tìm đường đi ngắn nhất trên đồ thị không trọng số (hoặc tất cả các cạnh có cùng trọng số), bằng cách duyệt theo từng lớp số cạnh tăng dần. Khi các cạnh có thời gian (trọng số) khác nhau, một đường đi có ít cạnh hơn hoàn toàn có thể có tổng thời gian lớn hơn một đường đi có nhiều cạnh nhưng mỗi cạnh có trọng số rất nhỏ. Trong trường hợp cạnh có trọng số thời gian khác nhau, bắt buộc phải sử dụng thuật toán Dijkstra (với trọng số không âm) hoặc Bellman-Ford.

Q3. Giải thích hai phương án tạo ô F(2,8) và vị trí của ô đó trong table Julia. Vì sao nhánh lấy đơn phải dùng hàng trước?

• Giải thích hai phương án tạo ô F(2, 8):

Xét đơn hàng thứ 2 (có chi phí $\tau_2 = 5$, lợi ích $v_2 = 8$) với ngân sách thời gian b = 8:

- Phương án 1 (Không lấy đơn 2): Giá trị tối ưu bằng giá trị tốt nhất khi chỉ xét 1 đơn đầu tiên với toàn bộ ngân sách 8: F(1, 8) = 6.
- Phương án 2 (Lấy đơn 2): Nhận lợi ích $v_2 = 8$ của đơn 2, cộng với giá trị tối ưu của 1 đơn đầu tiên với ngân sách còn lại ($8 - \tau_2 = 8 - 5 = 3$): $v_2 + F(1, 3) = 8 + 6 = 14$.
- Giá trị ô F(2, 8) là giá trị lớn hơn giữa hai phương án: $F(2, 8) = \max(F(1, 8), v_2 + F(1, 3)) = \max(6, 14) = 14$.

• Vị trí của ô F(2, 8) trong table Julia:

Do Julia dùng mảng chỉ số bắt đầu từ 1, có thêm hàng cơ sở i = 0 và cột cơ sở b = 0. Vì vậy, trạng thái F(i, b) được ánh xạ vào table[i+1, b+1]. Với i = 2, b = 8, vị trí chính xác là table[2+1, 8+1] = table[3, 9].

• Vì sao nhánh lấy đơn phải dùng hàng trước (i - 1):

Trong bài toán Ba lô 0/1, mỗi đơn hàng chỉ được phép chọn tối đa một lần (0 hoặc 1). Nếu nhánh lấy đơn tham chiếu đến cùng hàng i (tức F(i, b - τ_i)), thì trong ngân sách còn lại b - τ_i đã có thể chứa đơn i rồi, dẫn tới đơn i bị chọn lặp lại vô hạn lần (trở thành bài toán Ba lô không giới hạn). Việc bắt buộc lấy từ hàng trước (i - 1) đảm bảo ngân sách còn lại chỉ được tối ưu trên tập các đơn từ 1 đến i - 1, chắc chắn không chứa đơn i, duy trì tính đúng đắn của quyết định chọn đơn 0/1.

Q4. Sau khi giao ID 1 rồi hoàn tác ngay, FIFO, ngăn xếp, tổng thời gian và tổng lợi ích phải thay đổi thế nào? Nêu một bất biến kiểm tra việc không mất hoặc lặp đơn.

• Sự thay đổi của FIFO, ngăn xếp, tổng thời gian và tổng lợi ích khi giao ID 1 rồi hoàn tác ngay:

1. Trạng thái sau lập kế hoạch: Queue = [1, 2], Stack = [], spent = 0, earned = 0.
2. Khi thực hiện giao đơn ID 1 (deliver!):
   - FIFO: Đơn ID 1 ở đầu bị lấy ra (dequeue), hàng đợi còn lại: [2].
   - Ngăn xếp: ID 1 được đẩy vào đỉnh ngăn xếp (push), lịch sử thành: [1].
   - Tổng thời gian spent tăng thêm $\tau_1 = 3$ (spent = 3).
   - Tổng lợi ích earned tăng thêm $v_1 = 6$ (earned = 6).
3. Khi thực hiện hoàn tác ngay (undo!):
   - Ngăn xếp: Lấy đơn ID 1 ở đỉnh ngăn xếp ra (pop), lịch sử trở về: [].
   - FIFO: Đưa ID 1 trả lại vào ĐẦU hàng đợi (enqueue_front!), hàng đợi khôi phục lại: [1, 2].
   - Tổng thời gian spent giảm trừ $\tau_1 = 3$ (spent = 0).
   - Tổng lợi ích earned giảm trừ $v_1 = 6$ (earned = 0).

--> Trạng thái toàn hệ thống được khôi phục nguyên vẹn 100% như trước khi giao.

• Bất biến kiểm tra việc không mất hoặc lặp đơn:

Hợp của tập các đơn đang chờ trong hàng đợi (pending) và tập các đơn đã giao trong ngăn xếp (completed) luôn là hai tập rời nhau và bằng đúng tập các đơn đã được chọn trong kế hoạch (selected_ids):

$$Set(pending) \cap Set(completed) = \emptyset$$

$$Set(pending) \cup Set(completed) = Set(selected_ids)$$

Đồng thời: length(pending) + length(completed) == length(selected_ids). Bất biến này bảo đảm tuyệt đối rằng không có đơn hàng nào bị thất thoát hoặc bị nhân đôi trong suốt chu trình giao - hoàn tác.

Q5. Cho biết độ phức tạp theo n,V,E,B của sắp xếp trộn, tìm kiếm nhị phân, BFS cốt lõi và DP bảng đầy đủ. Vì sao không thể khẳng định mọi BST đều tra cứu trong O(log n)?

• Độ phức tạp theo n, V, E, B của các thuật toán:

1. Sắp xếp trộn (Merge Sort): Thời gian O(n log n), bộ nhớ phụ O(n) trong mọi trường hợp (tốt nhất, trung bình, xấu nhất).

2. Tìm kiếm nhị phân (Binary Search): Thời gian O(log n), bộ nhớ phụ O(1).

3. BFS cốt lõi: Thời gian O(V + E) trên danh sách kề, bộ nhớ phụ O(V) cho hàng đợi, mảng khoảng cách và nút cha.

4. Quy hoạch động ba lô (DP bảng đầy đủ): Thời gian O(n × B), bộ nhớ phụ O(n × B) cho bảng table (n là số đơn, B là giá trị ngân sách).

• Vì sao không thể khẳng định mọi BST đều tra cứu trong O(log n):

Thời gian tra cứu của Cây tìm kiếm nhị phân (BST) phụ thuộc tuyến tính vào chiều cao h của cây: O(h).

- Trường hợp lý tưởng: Khi cây BST cân bằng (hoặc xây dựng cân bằng từ trung điểm mảng đã sắp xếp), chiều cao cây đạt tối ưu $h = \lceil \log_2 n \rceil$, khi đó thời gian tra cứu là O(log n).

- Trường hợp suy biến: Nếu dữ liệu đầu vào được chèn vào cây theo thứ tự đã có thứ tự sẵn (tăng dần hoặc giảm dần, ví dụ chèn liên tiếp các ID: 1, 2, 3, 4, ...), cây BST thông thường không có cơ chế tự cân bằng sẽ bị lệch hoàn toàn về một phía, suy biến thành một danh sách liên kết đơn có chiều cao h = n. Lúc này, phép tra cứu một phần tử ở cuối cây đòi hỏi duyệt qua toàn bộ n nút, dẫn tới độ phức tạp thời gian trong trường hợp xấu nhất là O(n) chứ không phải O(log n).

Vì vậy, ta chỉ có thể khẳng định BST tra cứu trong O(log n) khi cây được đảm bảo cân bằng (như AVL, Red-Black Tree).